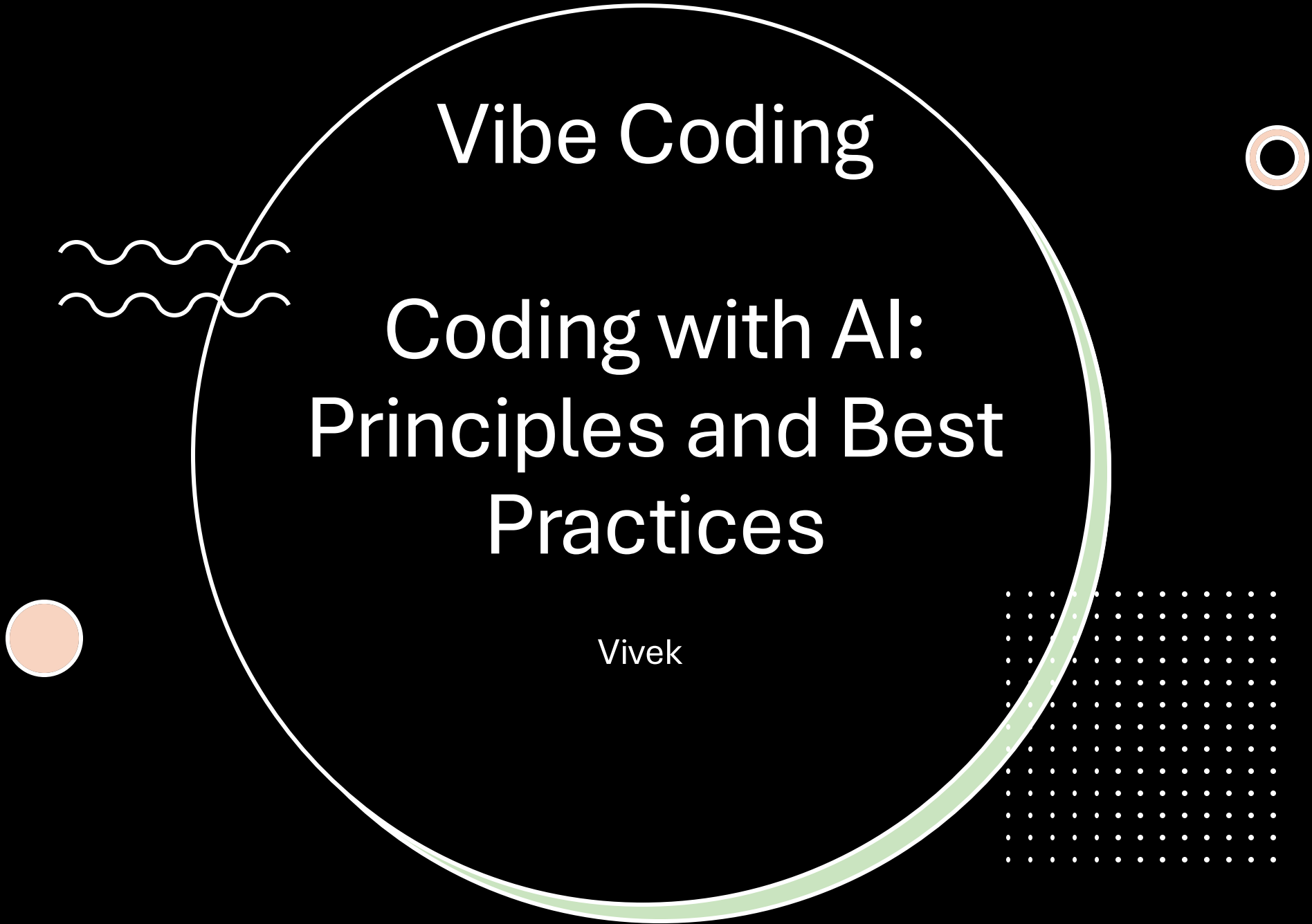


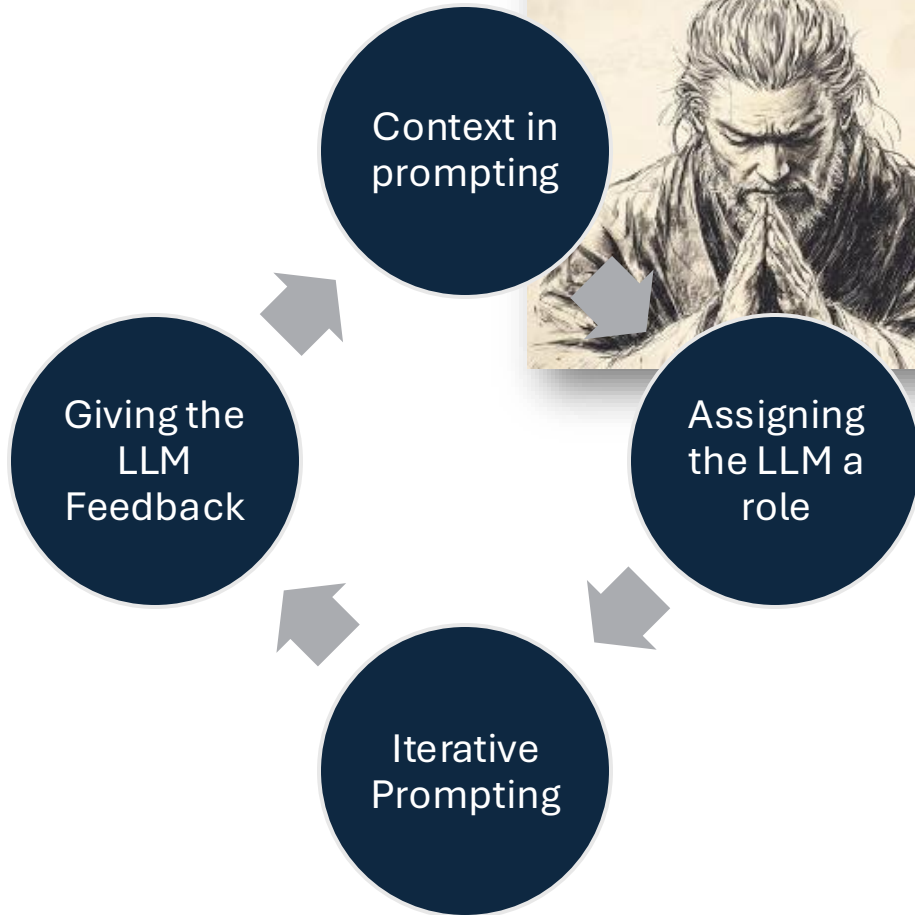


Vibe Coding

Coding with AI:
Principles and Best
Practices

Vivek

Principles



- **Be Specific:** Provide detail and context about your problem
- **Assign a role:** Assign a role to tailor the output you receive
- **Request an expert opinion:** Assign an expert role and ask the LLM to evaluate the work you've already done to further refine it
- **Give feedback:** Iteratively prompt the LLM and provide feedback on the output you receive to get closer to your expected results

Three Pillars

1

Specification: You define the goal (e.g., "Build a Twitter clone with login").

2

Rules: You set explicit constraints (e.g., "Use Python, avoid complexity").

3

Oversight: You monitor and steer the process to ensure alignment.

Why Is This Framework Useful?

Issues

Tames Chaos: Enforces strict adherence to your rules, minimizing runaway behavior.

Saves Time: Structured steps and summaries reduce rework.

Enables Clarity: Non-technical users can follow along; programmers gain precision.

Key Benefits

Clarity: Rules are modular, making them easy to navigate and adjust.

Control: You dictate the pace and scope of AI actions.

Scalability: Works for small scripts (e.g., a calculator) or large apps (e.g., a web platform).

Maintainability: Documentation and tracking ensure long-term project viability.

Coding Preferences – “Write code like this”



Purpose: Ensures clean, maintainable, and efficient code.



Rules:

Simplicity: "Always prioritize the simplest solution over complexity." No
Duplication: "Avoid repeating code; reuse existing functionality when possible."
Organization: "Keep files concise, under 200-300 lines; refactor as needed."
Documentation: "After major components, write a brief summary
in /docs/[component].md (e.g., login.md)."



Why It Works: Simple code reduces bugs; documentation provides a readable audit trail.

Principles: "Follow SOLID principles (e.g., single responsibility, dependency
inversion) where applicable."



Extensions:

Guardrails: "Never use mock data in dev or prod—restrict it to tests."

Context Check: "Begin every response with a random emoji (e.g., 🍌) to confirm
context retention."

Efficiency: "Optimize outputs to minimize token usage without sacrificing
clarity."



Technical Insight: SOLID ensures modularity (e.g., a login module doesn't handle tweets);
emoji signal when context exceeds model limits (typically 200k tokens for Claude 3.7).

Technical Stack – “Use These Tools”

- Purpose: Locks the AI to your preferred technologies.
- Rules (Berman’s Example):
 - "Backend in Python."
 - "Frontend in HTML and JavaScript."
 - "Store data in SQL databases, never JSON files."
 - "Write tests in Python."
- Why It Works: Consistency prevents AI from switching tools mid-project.
- Extensions:
 - "If I specify additional tools (e.g., Elasticsearch for search), include them here."
 - "Never alter the stack without my explicit approval."
- Technical Insight: A fixed stack prevents AI from introducing incompatible dependencies (e.g., switching SQL to JSON).

Workflow Preferences – “Work This Way”

Purpose: Controls the AI’s execution process for predictability.

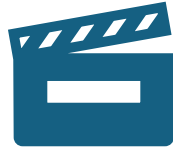
- Focus: "Modify only the code I specify; leave everything else untouched."
- Steps: "Break large tasks into stages; pause after each for my approval."
- Planning: "Before big changes, write a plan.md and await my confirmation."
- Tracking: "Log completed work in progress.md and next steps in TODO.txt."

Why It Works: Incremental steps and logs keep the process transparent and manageable.

Extensions:

- Testing: "Include comprehensive tests for major features; suggest edge case tests (e.g., invalid inputs)."
- Context Management: "If context exceeds 100k tokens, summarize into context-summary.md and restart the session."
- Adaptability: "Adjust checkpoint frequency based on my feedback (more/less granularity)."

Technical Insight: Token limits (e.g., Claude’s 200k) degrade performance beyond 100k; summaries maintain continuity. Tests catch regressions early.



Communication Preferences – “Talk to Me Like This”

- Purpose: Ensures clear, actionable feedback from the AI.
 - Summaries: "After each component, summarize what's done."
 - Change Scale: "Classify changes as Small, Medium, or Large."
 - Clarification: "If my request is unclear, ask me before proceeding."
- Why It Works: You stay informed without needing to decipher AI intent.
- Extensions:
 - Planning: "For Large changes, provide an implementation plan and wait for approval."
 - Tracking: "Always state what's completed and what's pending."
 - Emotional Cues: "If I indicate urgency (e.g., 'This is critical—don't mess up!'), prioritize care and precision."
- Technical Insight: Change classification (S/M/L) quantifies impact (e.g., Small = <50 lines, Large = architecture shift); emotional cues may leverage training data patterns for better compliance.

Practical Example: How It Works

Task: "Build a note-taking app with save functionality."

Specification: You say, "I want an app to write and save notes."

AI Response:

- "🦋 Understood. Plan: 1. Backend (Python, SQL storage), 2. Frontend (HTML/JS), 3. Save function. Proceed?"
- You: "Yes."

Execution:

- After backend: "👉 Backend done (Medium change). Notes saved in SQL. Updated progress.md and TODO.txt. Next: frontend?"
- After frontend: "🌟 Frontend complete. Added docs/notes.md with usage. Done!"

Outcome: A working app with logs (progress.md, /docs) for reference.

Technical Note: Each step is testable (e.g., SQL insert works), and context is preserved via summaries.

Advanced Tips: Maximizing the Framework

Why Four Files?

- Modularity: Each file isolates a concern—style, tools, process, communication—for easy updates.
- Scalability: Adjust one file without disrupting others (e.g., tweak communication without touching stack).

Customization Options

- Beginners: Skip advanced rules (e.g., SOLID) for simplicity.
- Teams: Add team-collaboration.mdc: "Align with team conventions in team-standards.md; summarize for peers."
- Large Projects: Increase checkpoints and documentation frequency.

Emotional Prompting

- Try: "This project is critical—please focus!" Anecdotal evidence suggests improved attention, possibly from training data biases.

Workflow

Plan & Design

- Requirement Analysis, Summary
- High Level - Arch, Sequence Diagrams, Algo & DS
- Low Level - DB Schema, ER, Data Dictionary, Models, API Contracts

Implement & Optimize

- DB, API, Views
- Individual Components, Integration, Feature Complete
- Performance, Security, Scalability

Test & Document

- Code Review
- Exploratory, Functional
- Documentation

Plan & Design

Implement & Optimize

Test & Document

- TDD is important because it forces the model to optimize for the test case - it becomes an easier matching problem when you have a constraint
- Always iterate on the discussion - pushing for more concise, minimalistic, idiomatic, and "adjective" solution
- Always play the rubber-duck debugging game - ask it to explain the logic - so that it can discover its own inconsistencies

Steps for building:

- Create a new folder for context retention called aa_ref_01
- Create file named r01_initial_ideation.txt for just noting down all the ideas without any structure
- Create file named r02_domain_analysis.txt which will contain the best of the conversation we have with tab nine or @web in order summarize the best domain knowledge needed for the problem statement
- Create file named r03_prd_v1.txt which will be the first PRD for the problem containing the user journey at a high level
- Create file named r04_prd_v2.txt which hopefully is a more precise version of the previous PRD based on multiple iterations on the previous file
- Create file named r05_arch_L1.txt which will contain the first attempt at the architecture of the PRD - this file should leverage a relationship of consistency with r04_prd_v2.txt + r02_domain_analysis.txt
- Create file named r06_arch_L2.txt which will contain the second iteration of the architecture of the PRD, this file should leverage a relationship of consistency with r05_arch_L1.txt + r04_prd_v2.txt + r02_domain_analysis.txt
- Create file named r07_explain_architecture.txt which will contain the explanation of the architecture of the PRD - this file should leverage a relationship of consistency with r06_arch_L2.txt + r04_prd_v2.txt + r02_domain_analysis.txt. This file is basically a rubber-duck debugging of the architecture with us as the listener who wants to make sure that the r06_arch_L2.txt is well thought out
- Create a file named r08_TDD_v1.txt which will basically ask the LLM to write a TDD plan to implement the r06_arch_L2.txt in the most minimalistic way possible, it should be consistent with r04_prd_v2.txt & r02_domain_analysis.txt
- If the r08_TDD_v1.txt is good enough, then select each step of the TDD plan as a checkbox - and ask the strong LLM (code LLM) to implement the TDD plan in the most idiomatic bug-free way possible line by line, thoroughly following the TDD methodology of starting with Red Test, then Green Test, then Refactor
- Create a file named r09_bug_log.txt to note copy all the bugs encountered in the process of implementing the TDD plan
- Create a file named r10_avoid_bugs.txt which will contain meta patterns and best practices discovered during the implementation of the TDD plan
- Create UAT plan for the whole feature, ideally this should be very close

Latest

- Do a two-step process, first only plan vs. code.
- Break tasks into one small micro step at a time
- Use Grok to plan and Sonnet 3.7 thinking to code

- the solution
- TDD is important because it forces the model to optimize for the test case - it becomes an easier matching problem when you have a constraint
- Always iterate on the discussion - pushing for more concise, minimalistic, idiomatic, and "adjective" solution
- Always play the rubber-duck debugging game - ask it to explain the logic - so that it can discover its own inconsistencies

Steps for building:

- Create a new folder for context retention called aa_ref_01
- Create file named r01_initial_ideation.txt for just noting down all the ideas without any structure
- Create file named r02_domain_analysis.txt which will contain the best of the conversation we have with tab nine or @web in order summarize the best domain knowledge needed for the problem statement
- Create file named r03_prd_v1.txt which will be the first PRD for the API containing the user journey at a high level
- Create file named r04_prd_v2.txt which hopefully is a more precise version of the previous PRD based on multiple iterations on the previous file
- Create file named r05_arch_L1.txt which will contain the first attempt at the architecture of the PRD - this file should leverage a relationship of consistency with r04_prd_v2.txt + r02_domain_analysis.txt
- Create file named r06_arch_L2.txt which will contain the second iteration of the architecture of the PRD, this file should leverage a relationship of consistency with r05_arch_L1.txt + r04_prd_v2.txt + r02_domain_analysis.txt
- Create file named r07_explain_architecture.txt which will contain the explanation of the architecture of the PRD - this file should leverage a relationship of consistency with r06_arch_L2.txt + r04_prd_v2.txt + r02_domain_analysis.txt. This file is basically a rubber-duck debugging of the architecture with us as the listener who wants to make sure that the r06_arch_L2.txt is well thought out
- Create a file named r08_TDD_v1.txt which will basically ask the LLM to write a TDD plan to implement the r06_arch_L2.txt in the most minimalistic way possible, it should be consistent with r04_prd_v2.txt & r02_domain_analysis.txt
- If the r08_TDD_v1.txt is good enough, then select each step of the TDD plan as a checkbox - and ask the strong LLM (code LLM) to implement the TDD plan in the most idiomatic bug-free way possible line by line, thoroughly following the TDD methodology of starting with Red Test, then Green Test, then Refactor
- Create a file named r09_bug_log.txt to note copy all the bugs encountered in the process of implementing the TDD plan
- Create a file named r10_avoid_bugs.txt which will contain meta patterns and best practices discovered during the implementation of the TDD plan
- Create UAT plan for the whole feature, ideally this should be very close

Getting the most out of LLMs

Experiment with what you LLM can do



Test LLM code carefully



Use LLMs as a learning tool



You are the context expert



References

Thank You!
Questions?

- Course link: [Generative AI for Software Development](#)
- Andrej Karpathy: Coined the term "vibe coding" and introduced it to the broader community in a post on X (February 3, 2025, <https://x.com/karpathy/status/1886192184808149383>)
- [Matthew Berman: Core vibe coding rules and philosophy \(YouTube, 2025\).](#)
- <https://github.com/PatrickJS/awesome-cursorrules>
- <https://cursor.directory/>
- <https://x.com/petergyang/status/1900727143417737612>
- **YouTube Community:**
 - [u/nufh](#), [u/believablybad](#) (documentation, .md files).
 - [u/robistocco](#) (iterative workflow).
 - [u/xmontc](#) (checkpoints).
- **Reddit Community:**
 - [u/illusionst](#) (communication, progress tracking).
 - [u/Puzzleheaded-Age-660](#) (token optimization).
 - [u/DonkeyBonked](#), [u/philip_laureano](#) (KISS, DRY, YAGNI, SOLID).
 - [u/evia89](#) (emoji context check).
 - [u/dhamaniasad](#), [u/capecoderrr](#) (emotional prompting).
- https://www.reddit.com/r/ChatGPTCoding/comments/1j5l4xw/vibe_coding_manual/